Grey Wolf Optimizer

Image Processing

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Parameters
nPop = 10
max_iter = 30
NB = 2                        # Number of features to select
nFeatures = X.shape[1]

# Calculate category separability (Fisher Score)
scores = []

for j in range(nFeatures):
    overall_mean = np.mean(X[:, j])
    between = 0
    within = 0

    for c in np.unique(y):
        values = X[y == c, j]
        between += len(values) * (np.mean(values) - overall_mean) ** 2
        within += np.sum((values - np.mean(values)) ** 2)

    scores.append(between / (within + 1e-10))

# Initialize population using category separability
population = np.random.rand(nPop, nFeatures)
```

```python
for i in range(nPop):
    population[i] += np.array(scores)

# Objective function
def fitness(position):
    selected = np.argsort(position)[-NB:]

    X_selected = X[:, selected]

    X_train, X_test, y_train, y_test = train_test_split(
        X_selected, y, test_size=0.3, random_state=42
    )

    model = SVC(kernel='rbf')
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)

    return accuracy

# HGWO Algorithm
for t in range(max_iter):

    # Calculate fitness
    fit = np.array([fitness(w) for w in population])

    # Find Alpha, Beta and Delta
    order = np.argsort(fit)[::-1]

    alpha = population[order[0]].copy()
    beta = population[order[1]].copy()
    delta = population[order[2]].copy()
```

```
    alpha = population[order[0]].copy()
    beta = population[order[1]].copy()
    delta = population[order[2]].copy()

    # Nonlinear convergence factor
    a = 2 - 2 * (np.exp(t / max_iter) - 1) / (np.e - 1)

    # Update wolves
    for i in range(nPop):
        for j in range(nFeatures):

            r1, r2 = np.random.rand(), np.random.rand()
            A1 = 2 * a * r1 - a
            C1 = 2 * r2

            D_alpha = abs(C1 * alpha[j] - population[i, j])
            X1 = alpha[j] - A1 * D_alpha

            r1, r2 = np.random.rand(), np.random.rand()
            A2 = 2 * a * r1 - a
            C2 = 2 * r2

            D_beta = abs(C2 * beta[j] - population[i, j])
            X2 = beta[j] - A2 * D_beta

            r1, r2 = np.random.rand(), np.random.rand()
            A3 = 2 * a * r1 - a
            C3 = 2 * r2

            D_delta = abs(C3 * delta[j] - population[i, j])
            X3 = delta[j] - A3 * D_delta

            # Update position
            population[i, j] = (X1 + X2 + X3) / 3
```

```
            D_delta = abs(C3 * delta[j] - population[i, j])
            X3 = delta[j] - A3 * D_delta

            # Update position
            population[i, j] = (X1 + X2 + X3) / 3

# Find best solution
fit = np.array([fitness(w) for w in population])

best = population[np.argmax(fit)]

selected = np.argsort(best)[-NB:]

print("Selected Features:", selected)
print("Best Classification Accuracy:", max(fit) * 100, "%")
```

Output:

```
Selected Features: [2 3]
Best Classification Accuracy: 100.0 %
```